

Cahier des charges — version détaillée et simplifiée

Outil de suivi et d'analyse de l'activité de formation — Galaxy Solutions

1. Contexte & objectif

Galaxy Solutions est une entreprise qui **forme les salariés d'autres entreprises (des PME)**. Elle propose des formations sur 3 grands domaines :

- **Technologies web et data** (ex : Python, SQL, développement web)
- **Management agile** (ex : Scrum, gestion de projet)
- **Cybersécurité**

Chaque formation dure entre **2 et 5 jours**, et peut se donner de deux façons :

- **En intra-entreprise** : une seule entreprise cliente réserve toute la session pour ses propres salariés.
- **En inter-entreprises** : plusieurs entreprises différentes envoient chacune quelques salariés à une même session commune.

Le problème actuel : toutes les informations (qui est inscrit où, quel formateur anime quoi, quel client a réservé quoi) sont dispersées dans des fichiers Excel séparés, gérés probablement par différentes personnes. Résultat :

- Pas de base de données centralisée
- Pas de vision d'ensemble de l'activité
- Impossible de répondre facilement à des questions simples comme "combien de sessions cybersécurité ce trimestre ?" ou "quel est notre client le plus actif ?"

Ta mission : construire un outil interne qui règle ce problème, en deux temps :

1. Centraliser toutes ces données dans une vraie base de données
2. Calculer et afficher des indicateurs fiables pour aider les responsables à piloter l'activité

La priorité absolue, dite explicitement dans le cahier des charges : un socle qui fonctionne et des indicateurs justes comptent plus que des fonctionnalités avancées mais bancales. Mieux vaut un outil simple qui marche à 100% qu'un outil ambitieux à moitié fini.

2. Périmètre fonctionnel & technique

Le travail est organisé en **3 niveaux progressifs**. Règle d'or : **on ne passe au niveau suivant que si le niveau précédent est fini, stable, et testé**. Pas de niveau 2 commencé si le niveau 1 a encore des bugs.

Niveau 1 — Obligatoire (le socle)

C'est la fondation. Sans ça, il n'y a pas de projet. Il comprend :

a) Modéliser et créer la base de données, avec au minimum ces entités :

- **Formations** : le catalogue des formations proposées (ex : "Python niveau débutant", domaine = web & data)
- **Sessions** : une instance précise et planifiée d'une formation (ex : "Python débutant, du 5 au 7 août 2026")
- **Inscriptions** : le lien entre un salarié et une session à laquelle il participe
- **Formateurs** : les personnes qui animent les sessions
- **Clients** : les entreprises qui inscrivent leurs salariés
- **Utilisateurs** : les comptes qui se connectent à l'application, chacun avec un rôle

b) Développer le backend avec Flask (framework Python) et SQLAlchemy (pour parler à la base de données). Il faut pouvoir :

- Créer de nouvelles entrées (ex : ajouter une session)
- Consulter les données existantes (ex : voir la liste des inscrits)
- Mettre à jour les données (ex : modifier les dates d'une session)

c) Gérer trois rôles utilisateurs, chacun avec des droits différents :

- **Administrateur** : contrôle total, y compris la gestion des comptes utilisateurs
- **Gestionnaire** : gère le quotidien (formations, sessions, inscriptions, clients)
- **Formateur** : accès limité à ses propres sessions

d) Générer un jeu de données de démonstration réaliste, c'est-à-dire un script qui remplit automatiquement la base avec des fausses données crédibles : plusieurs sessions de durées variées (2 à 5 jours), réparties sur les 3 domaines, avec plusieurs clients et formateurs différents. Ce jeu de données sert à tester que tout fonctionne, y compris les calculs de statistiques plus tard.

e) Exposer une API REST, c'est-à-dire un ensemble de "portes d'entrée" (endpoints) qui permettent à un programme extérieur (ou à ton interface) de récupérer ou modifier les données via des requêtes HTTP (GET, POST, PUT...). Cette API doit être **documentée** — un simple README qui explique chaque endpoint suffit, pas besoin d'un Swagger complet si tu manques de temps.

Niveau 2 — Souhaitable (uniquement si le niveau 1 est stable)

a) Développer des requêtes analytiques, c'est-à-dire des calculs sur les données :

- **Taux de remplissage** des sessions (nombre d'inscrits ÷ capacité maximale)

- **Activité par domaine** (combien de sessions/inscriptions en cybersécurité, agile, web & data)
- **Activité par client** (quelle entreprise consomme le plus de formations)
- **Activité par formateur** (qui anime le plus de sessions, dans quels domaines)
- **Évolution des inscriptions dans le temps** (tendance mois par mois)

b) Restituer ces indicateurs dans un tableau de bord : une interface visuelle (graphiques, tableaux) qui affiche ces chiffres de façon lisible pour un responsable Galaxy Solutions.

Niveau 3 — Optionnel (seulement si niveau 1 et 2 sont impeccables avant la fin de la semaine 3)

a) Analyse multivariée (ACP) : une technique statistique pour regrouper automatiquement des formations ou des clients qui se ressemblent (par exemple, des clients qui consomment des profils de formation similaires).

b) Composant d'aide à la décision : un système simple qui, par exemple, alerte automatiquement quand une session risque de ne pas être assez remplie, ou qui recommande des formations à un client selon son historique.

Règle stricte : ces deux extensions ne doivent jamais être commencées au détriment du socle. Si le socle n'est pas stable fin de semaine 3, on ne les commence pas, point final.

Cadre technique imposé

Élément	Choix
Backend	Python, Flask, SQLAlchemy
Analyse de données	Pandas, NumPy
Base de données	MySQL ou SQLite (au choix)
Frontend	Libre, selon le temps disponible (voir garde-fou)
Versionnement	Git / GitHub, avec des commits réguliers

Le garde-fou frontend (règle importante à retenir)

Le développement de l'interface (frontend) **ne doit jamais prendre le pas sur le backend**. Si faire une interface en JavaScript prend trop de temps, il faut livrer une interface simple avec des templates Jinja2 (le système de templates intégré à Flask).

Principe clé : une interface sobre sur un backend qui fonctionne vaut toujours mieux qu'un joli dashboard sur un backend incomplet ou buggé. Ce qui compte avant tout : l'API doit tourner et les calculs analytiques doivent être exacts.

3. Planning (4 semaines)

Semaine 1 — Conception et données

- Rencontrer le tuteur pour cadrer précisément le besoin et figer ce qui rentre dans le socle
- Produire le **diagramme de classes** et le **modèle logique de données (MLD)** — le schéma qui décrit toutes les tables et leurs relations
- Créer la base de données et générer le jeu de données de démonstration

Semaine 2 — Backend et API

- Développer les entités (les modèles SQLAlchemy) et la logique métier associée
- Mettre en place les trois rôles utilisateurs (authentification, permissions)
- Créer et documenter les endpoints de l'API REST
- Tester chaque endpoint au fur et à mesure qu'il est créé (pas tout à la fin)

Semaine 3 — Indicateurs et restitution

- Développer les requêtes analytiques (les calculs de statistiques)
- Restituer ces indicateurs dans le tableau de bord (ou dans l'interface Jinja2 si le garde-fou s'applique)
- Vérifier que les chiffres calculés sont exacts en les comparant au jeu de données de démonstration (pas juste "ça s'affiche" — "le calcul est juste")

Semaine 4 — Extensions ou consolidation, puis clôture

- **Si le socle est impeccable** : démarrer une extension optionnelle (ACP ou aide à la décision)
- **Sinon** : consolider l'existant, corriger les bugs, fiabiliser le socle et les indicateurs
- Rédiger la documentation technique et le rapport de stage

4. Livrables finaux

Ce qu'il faudra rendre à la fin du stage :

1. **Code source complet**, versionné sur GitHub avec un historique de commits
2. **Base de données** + le script qui génère le jeu de données de démonstration
3. **API REST fonctionnelle et documentée** (README ou spécification Swagger/OpenAPI)
4. **Diagramme de classes et modèle logique de données (MLD)**
5. **Tableau de bord** ou interface de restitution des indicateurs
6. **Documentation technique courte** : comment installer le projet, le lancer, et comment il est structuré
7. **Rapport de stage**

5. Critères d'évaluation

Critère	Ce qui est vérifié concrètement
Robustesse	Le backend démarre sans erreur, l'API répond correctement, les endpoints ont été testés
Justesse	Les indicateurs calculés sont corrects et vérifiables (pas juste plausibles — vraiment exacts)
Qualité du code	Code lisible, bien organisé, commits réguliers, gestion propre des erreurs
Pertinence	Les indicateurs choisis répondent à un vrai besoin de pilotage pour Galaxy Solutions
Autonomie	Respect des priorités données (socle avant souhaitable avant optionnel), qualité des choix faits en cas d'arbitrage
Documentation	Une personne qui ne connaît pas le projet peut l'installer, le lancer et le comprendre à partir de la doc seule
